DO NOW — RECALL

RISC and CISC

Name: _____ Class: _____ Date: _____

1. Why can instruction fetch and data access contend in Von Neumann architecture?

2. What is a pipeline hazard?

3. What is an opcode?

4. Array [2,4,6,8,10,12,14,16,18], target 16. Give midpoint indexes inspected with low=0 high=8 and integer midpoint.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

They share a memory pathway or resource.

2. Model answer

An event such as a dependency, branch or memory stall disrupting instruction overlap.

3. Model answer

The operation part of a machine or assembly instruction.

4. Model answer

mid4 value10 → low5 high8, mid6 value14 → low7 high8, mid7 value16 → found. Indexes 4,6,7.

Vocabulary — know these terms

RISC — Reduced Instruction Set Computer design approach.

CISC — Complex Instruction Set Computer design approach.

load/store — Access memory through dedicated instructions.

code density — Amount of program function per unit of machine-code storage.

opcode — Operation part of an instruction.

NEW LEARNING

Read, annotate and explain

RISC approach

Typically favours a smaller set of comparatively simple, regular instructions, often using explicit load/store operations.

CISC approach

Can offer complex instructions that express several low-level operations in one instruction, potentially improving code density.

Avoid oversimplifying

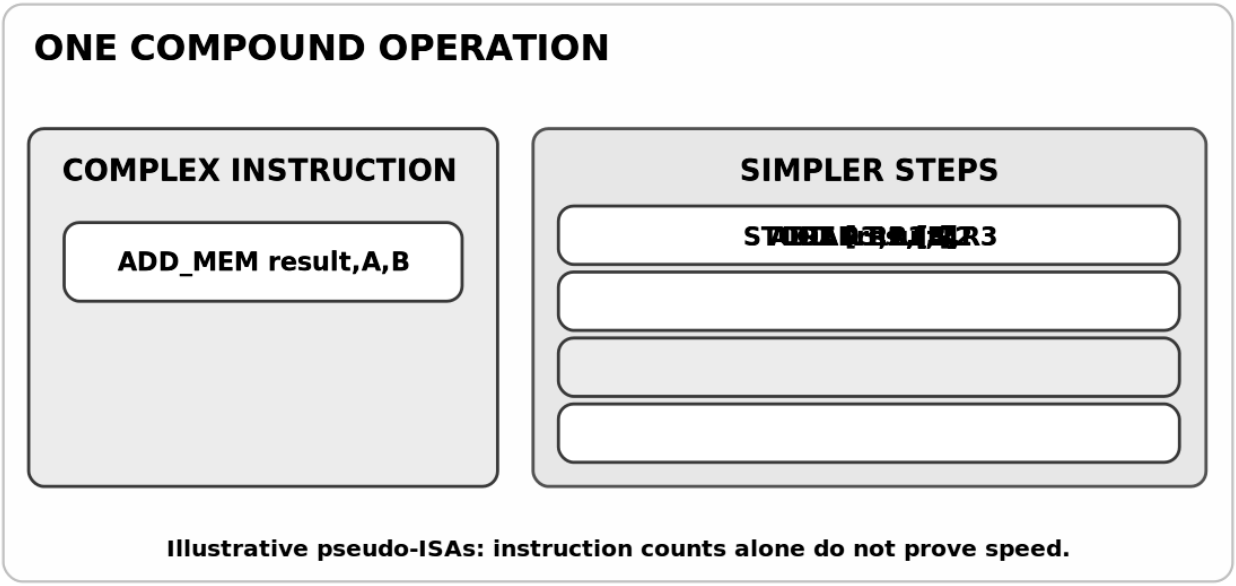
Fewer instructions in assembly do not guarantee fewer clock cycles. Actual performance depends on pipelining, decoding, caching and processor design.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

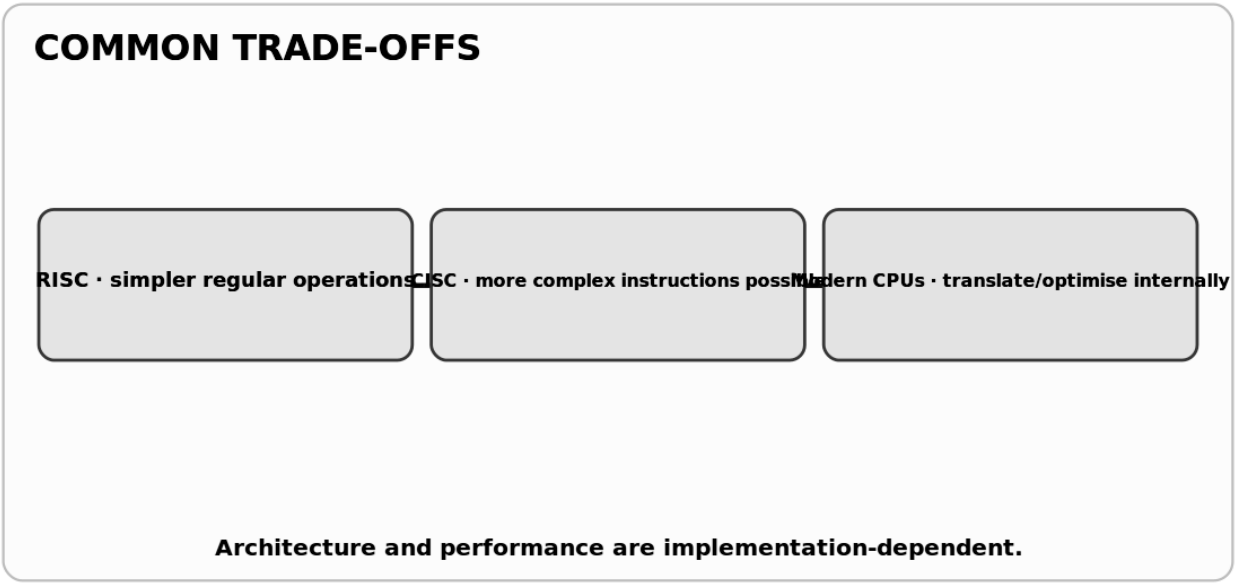
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



ONE COMPOUND OPERATION: Illustrative pseudo-ISAs: instruction counts alone do not prove speed.

New Learning



COMMON TRADE-OFFS: Architecture and performance are implementation-dependent.

I DO — WORKED EXAMPLE

Follow the worked method

Explain why a load/store architecture needs separate load and add instructions.

Predict first

Values are brought from memory into registers before the arithmetic instruction.

Explain and check

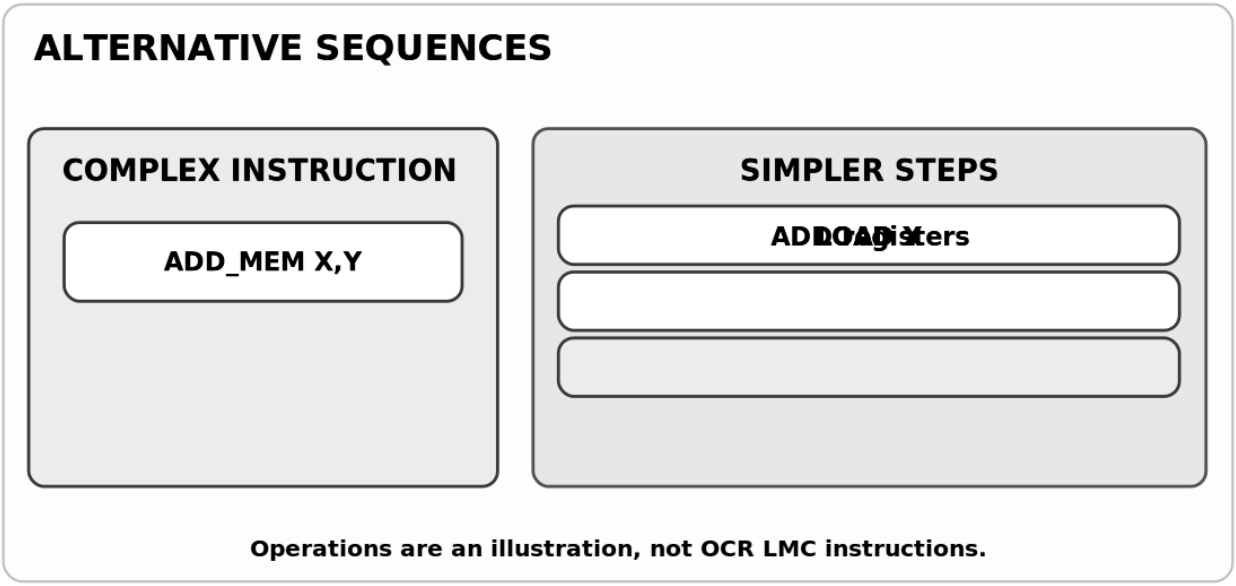
A regular load/store instruction set simplifies certain datapaths but may need more assembly instructions.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Explain why a load/store architecture needs separate load and add instructions.

Values are brought from memory into registers before the arithmetic instruction.



ALTERNATIVE SEQUENCES: Operations are an illustration, not OCR LMC instructions.

WE DO — GUIDED PRACTICE

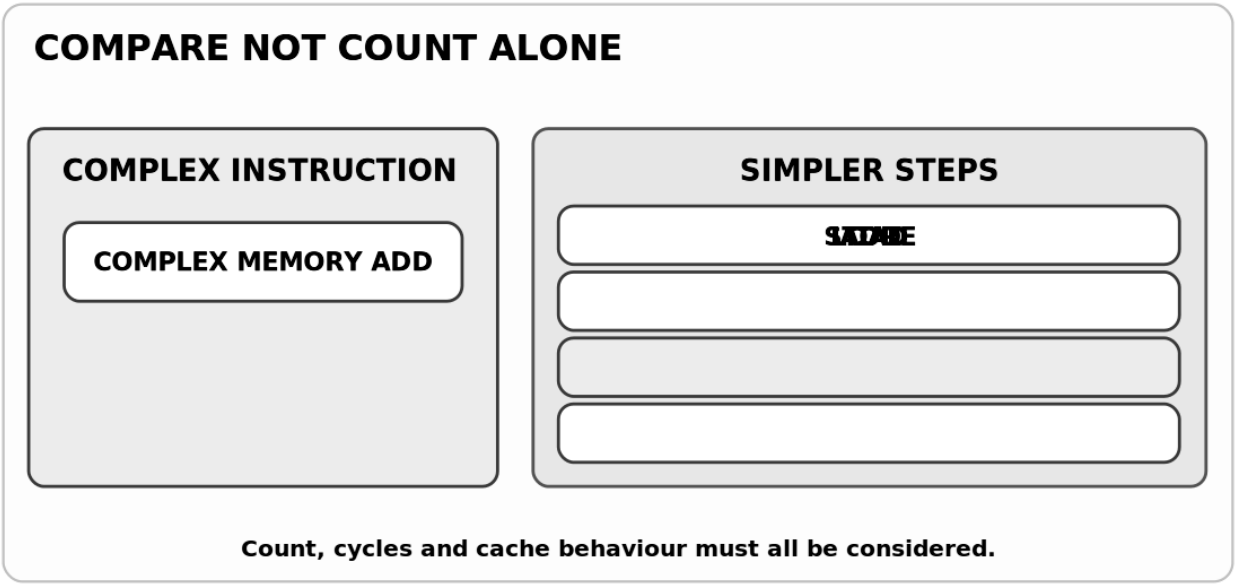
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Two instruction sets compute and store A+B. One offers a single memory-add; the other requires four steps. Doe

The complex ISA version uses fewer listed instructions but could require more work per instruction.



COMPARE NOT COUNT ALONE: Count, cycles and cache behaviour must all be considered.

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Predict first The complex ISA version uses fewer listed instructions but could require more work per instruction.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

State two characteristics of RISC and two of CISC.

Question 2

Compare one CISC-style memory-add instruction against a RISC-style LOAD, LOAD, ADD, STORE sequence for the same task.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Evaluate the claim 'RISC is always faster because its instructions are simpler'. Include code density, decoding and pipelining.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A program written using fewer assembly instructions always executes faster.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Explain and check

Evaluate clock cycles, decode overhead, code size, memory behaviour and hardware rather than choosing by instruction count alone.

IMPROVE: Explain one correction to your own reasoning.